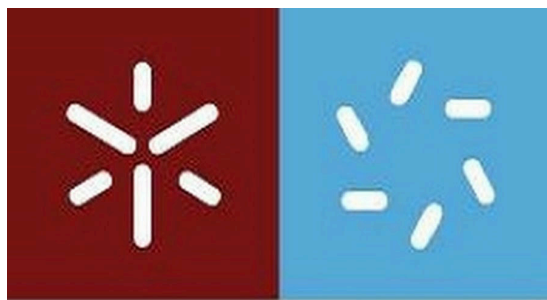




**Universidade do Minho**  
Escola de Ciências

## **Computação Gráfica**

2024/2025

### **Trabalho Prático**

#### **Fase 3 - Curvas, Superfícies Cúbicas e VBO's**

##### **Grupo 12**

Afonso Martins nº A102940

Luis Silva nº A105530

Pedro Gomes nº A102929

Ricardo Vilaça nº A102879

## Contents

|                                                               |   |
|---------------------------------------------------------------|---|
| 1. Introdução .....                                           | 3 |
| 2. Alterações ao gerador .....                                | 3 |
| 2.1. Leitura do ficheiro .patch .....                         | 3 |
| 2.2. Criação da superfície cúbica de Bezier .....             | 3 |
| 3. Alterações à engine .....                                  | 4 |
| 3.1. Alterações à estrutura de dados .....                    | 4 |
| 3.2. Translações dirigidas por uma curva de Catmull-Rom ..... | 5 |
| 3.3. Rotações dependentes do tempo .....                      | 7 |
| 3.4. Implementação de VBO's .....                             | 7 |
| 4. Sistema Solar Animado .....                                | 7 |
| 5. Rotação diária (Extra) .....                               | 8 |
| 6. Conclusão .....                                            | 8 |

# 1. Introdução

Nesta fase do projeto foi pedido que se modificasse as translações e as rotações aceites pela engine de forma a que fosse possível criar uma translação que seguisse uma curva de Catmull-Rom e uma rotação contínua que roda o modelo 360° em  $t$  tempo num certo eixo. Foi ainda pedido que o gerador fosse capaz de criar novas primitivas sobre a forma de Bezier *patches*. Por fim alterou-se o modo de desenhar as figuras, optando-se por utilizar VBO's para o efeito.

## 2. Alterações ao gerador

De forma a criar uma Bezier *patch*, é passado um ficheiro .patch ao gerador e ainda o nível de *tessellation* a utilizar para esta Bezier *patch*.

### 2.1. Leitura do ficheiro .patch

Para começar a criação da Bezier *patch* lê-se primeiro o ficheiro .patch passado ao gerador. Este ficheiro encontra-se no formato:

- Número de *patches*
- Índice dos pontos de cada *patch*
- Número total de pontos de controlo
- Pontos de controlo

Ao obter o número de patches que compõe a Bezier *patch*, cria-se um **vector<vector<int>>** que armazena os 16 pontos que definem cada patch. De seguida lêem-se os pontos de controlo, que são armazenados num **vector<vector<float>>**. Após a leitura de todos os pontos, fecha-se o ficheiro .patch e utiliza-se estes dois vetores para a criação da superfície cúbica de Bezier.

### 2.2. Criação da superfície cúbica de Bezier

Para a criação da superfície cúbica de Bezier, é necessário calcular os pontos que a compõe usando a seguinte equação:

$$p(u,v) = U * B * P * B^T * V$$

Sendo  $U = [u^3 \ u^2 \ u \ 1]$ ,  $B = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 1 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$ ,  $P = \begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{bmatrix}$ ,  $B^T = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 1 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$

e  $V = \begin{bmatrix} v^3 \\ v^2 \\ v \\ 1 \end{bmatrix}$

A matrix P representa os 16 pontos que definem o *patch* que está a ser utilizado. O número de pontos a ser calculado é  $N*N$  sendo N o nível de *tessellation* dado nos parâmetros de inicialização do gerador. Para calcular todos os pontos de uma *patch* usamos dois ciclos for, um para iterar sobre o  $u$  e outro para iterar sobre o  $v$ . Como cada célula da matrix P é um triplo  $(x,y,z)$  foi necessário separa-la em três matrizes diferentes, cada uma correspondente a uma das componentes do triplo, para se tornar mais fácil o cálculo de cada ponto.

$$p(u,v) = U * B * \begin{bmatrix} y_{P_{00}} & y_{P_{01}} & y_{P_{02}} & y_{P_{03}} \\ y_{P_{10}} & y_{P_{11}} & y_{P_{12}} & y_{P_{13}} \\ y_{P_{20}} & y_{P_{21}} & y_{P_{22}} & y_{P_{23}} \\ y_{P_{30}} & y_{P_{31}} & y_{P_{32}} & y_{P_{33}} \end{bmatrix} * B^T * V$$

A equação definida em cima exemplifica o cálculo da componente  $y$  do ponto  $p(u,v)$ , usando-se uma equação similar para calcular as outras duas componentes.

Com o ficheiro teste fornecido, foi possível criar um teapot representado na imagem abaixo:

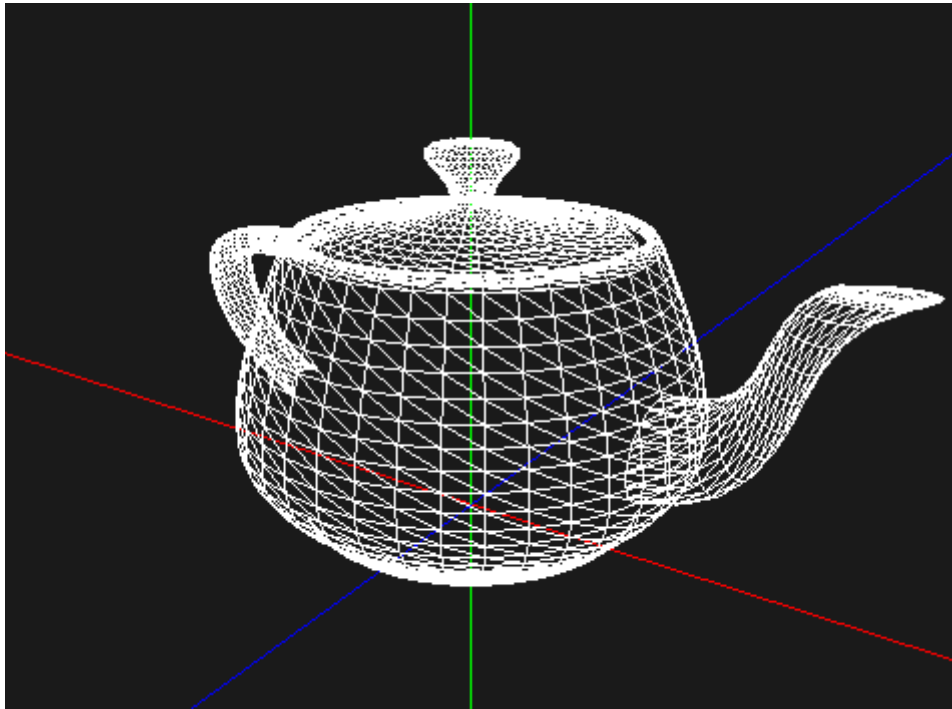


Figure 1: Teapot desenhado com Bezier *patches*

### 3. Alterações à engine

Foi também necessário alterar a forma como são processadas as rotações e as translações. Agora é também possível definir uma translação que segue e completa uma curva de Catmull-Rom em  $t$  tempo. É também possível definir uma rotação por tempo, ou seja, uma rotação contínua a volta de um eixo que completa  $360^\circ$  em  $t$  tempo. Para tal, foi necessário alterar a estrutura de dados onde são guardadas as transformações. Por fim alteramos o formato de desenho das figuras para VBO's para ter melhor performance.

#### 3.1. Alterações à estrutura de dados

Para além de alterar a estrutura *Transformation*, criou-se também uma nova estrutura de dados chamada *Point* que descreve um ponto no espaço e é composta por 3 atributos:

```
struct Point {
    float x, y, z;
};
```

Figure 2: Estrutura de dados 'Point'

À estrutura *Transformation* foram adicionados 3 atributos:

- um **float** *time* que define o tempo que uma rotação completa demora ou o tempo que demora a percorrer uma curva de Catmull-Rom;
- um **bool** *align* que define se um dado modelo fica alinhado à curva de Catmull-Rom;
- e ainda uma lista do tipo *Point* que guarda os pontos que descrevem a curva de Catmull-Rom;

```
struct Transformation {
    Type type;
    float x, y, z, angle = 0, time;
    bool align;
    list<Point> points;
};
```

Figure 3: Estrutura de dados ‘Transformation’

### 3.2. Translações dirigidas por uma curva de Catmull-Rom

A fim de executar animações em translações, estas devem receber um conjunto de pontos de controlo para definir a curva cúbica de Catmull-Rom e o tempo em segundos que o modelo deve demorar para percorrer toda a curva. Pode também ser indicado se o objeto deverá estar alinhado à curva através de um valor *align*.

Assim, para calcular os pontos da curva foram criadas duas funções a *getGlobalCatmullRomPoint* e *getCatmullRomPoint*. Para obter a proporção do instante de tempo atual, com o tempo da curva, recorre-se à seguinte fórmula, onde **time** é o tempo fornecido:

$$gt = (\text{glutGet}(\text{GLUT\_ELAPSED\_TIME}) / 1000.0) / \text{time}$$

A função *getGlobalCatmullRomPoint* serve para calcular, a um dado instante de tempo, a posição de um certo objeto e devolve os pontos da posição e da derivada da posição. Esta recolhe os quatro pontos de controlo da curva para o valor de tempo no momento e fornece-os à função *getCatmullRomPoint*.

A função *getCatmullRomPoint* está encarregue de calcular então a posição do ponto na curva de Catmull-Rom. Com os quatros pontos de controlo fornecidos,  $P$ , podemos determinar a sua posição através da formula seguinte:

$$p(t) = [t^3, t^2, t, 1] \begin{bmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1 & -2.5 & 2 & 0.5 \\ -0.5 & 0 & 0.5 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}$$

A derivada é obtida através da equação que se segue:

$$p'(t) = [3t^2 \ 2t \ 1 \ 0] \begin{bmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1 & -2.5 & 2 & 0.5 \\ -0.5 & 0 & 0.5 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix}$$

Caso o campo *align* seja aplicável a uma translação, precisamos definir a orientação dos eixos para alinhar à curva. Esta orientação é calculada com as fórmulas:

$$\vec{X}_i = p'(t)$$

$$\vec{Z}_i = X_i \times \vec{Y}_{i-1}$$

$$\vec{Y}_i = \vec{X}_i \times \vec{Z}_i$$

Assumi-mos a inicialização do eixo  $Y_0$  como o vetor de coordenadas (0,1,0). Nestas fórmulas  $p'(t)$  representa a derivada da curva no instante  $t$ , ' $\times$ ' o produto vetorial e  $Y_{i-1}$  a orientação do eixo Y no instante anterior ao atual. Com os cálculos podemos então determinar uma matriz de rotação de forma a alinhar o objeto à curva:

$$M = \begin{bmatrix} X_x & Y_x & Z_x & 0 \\ X_y & Y_y & Z_y & 0 \\ X_z & Y_z & Z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Para visualizar a linha das trajetórias das curvas, adicionamos também a função `renderCatmullRomCurve` que recebe os pontos de controlo da translação e calcula as posições de todos os pontos da curva de 0.01 em 0.01 segundos através da `getGlobalCatmullRomPoint`. Caso o tempo não seja indicado nas configurações XML, esse é considerado como 0 e a translação vai ser estática, isto é, sem animação. Toda a implementação das translações encontra-se na seguinte figura:

```
for (Transformation transform : group.transformations){
    if (transform.type == TRANSLATE){
        if (transform.time == 0){
            glTranslatef(transform.x, transform.y, transform.z);
        } else {
            float pos[3], deriv[3];
            float gt = ( glutGet(GLUT_ELAPSED_TIME) / 1000.0) / transform.time;

            getGlobalCatmullRomPoint(gt,transform.points,pos,deriv);
            renderCatmullRomCurve(transform.points);

            glTranslatef(pos[0], pos[1], pos[2]);

            if(transform.align){
                normalize(deriv);

                float z[3];
                cross(deriv, prev_y, z);
                normalize(z);

                float y[3];
                cross(z, deriv, y);
                normalize(y);

                prev_y[0] = y[0];
                prev_y[1] = y[1];
                prev_y[2] = y[2];

                float m[16];
                buildRotMatrix(deriv, y, z, m);

                glMultMatrixf(m);
            }
        }
    }
}
```

Figure 4: Translações com time

### 3.3. Rotações dependentes do tempo

Para obtermos uma rotação animada é necessário fornecer o tempo que o objeto demora a realizar uma rotação de 360 graus sobre o eixo pretendido. A partir desse valor, podemos deduzir o ângulo de rotação a qualquer instante da animação.

A todos os momentos da execução, calculamos o ângulo de rotação tendo em conta o tempo decorrido desde do início de execução do programa, obtido através do função `glutGet(GLUT_ELAPSED_TIME)` e do tempo de rotação definido na configuração XML. Assim, determinamos a seguinte fórmula, sendo o  $t$  o tempo decorrido e  $time$  o tempo definido para a rotação.

$$\text{angle} = 360^\circ * (t * 1000) / time$$

Visto que os ângulos não devem, ou podem, ultrapassar os 360°, sempre que acontece isso retira-se 360 ao resultado.

### 3.4. Implementação de VBO's

Para melhorar a performance da simulação alteramos o modo de desenho para VBO's. Para tal efeito, criamos uma variável *buffers* onde são guardados os vértices de cada modelo quando se itera sobre o ficheiro XML inicialmente. Posteriormente, na função de *display*, são aplicadas todas as transformações correspondentes e acedemos aos *buffers* de cada modelo para poderem ser desenhados.

## 4. Sistema Solar Animado

Para esta fase criamos uma sistema solar em que todos os planetas e satélites

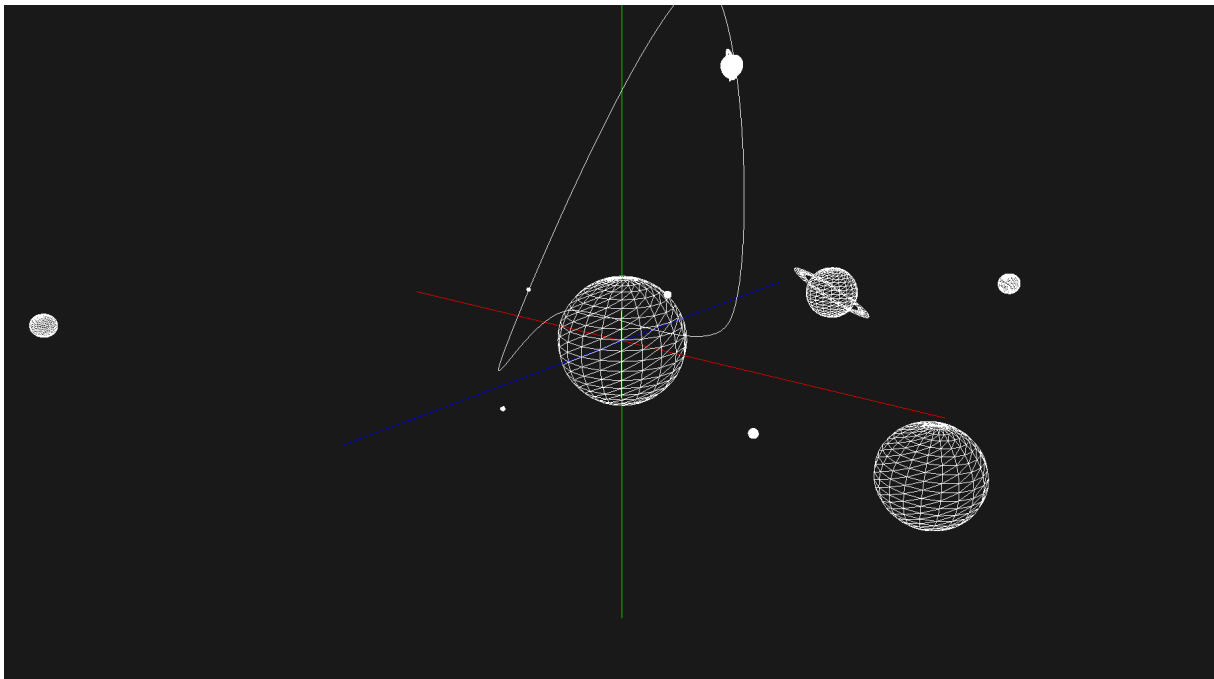


Figure 5: Sistema Solar

## 5. Rotação diária (Extra)

Para imitar a rotação dos planetas foi adicionado um campo opcional em cada modelo chamado *dayPeriod* onde é possível definir o tempo que o modelo demora a dar uma volta completa sobre o vetor *up*.

```
<!-- Terra -->
<group>
  <transform>
    <rotate time="13" x="0" y="1" z="0" />
    <translate x="-60" y="0" z="0" />
    <scale x="1.4" y="1.4" z="1.4"/>
  </transform>
  <models>
    <model file="sphere.3d" dayPeriod="10" />
  </models>

  <!-- Lua -->
  <group>
    <transform>
      <rotate time="10" x="0" y="1" z="0" />
      <translate x="3" y="0" z="2" />
      <scale x="0.27" y="0.27" z="0.27" />
    </transform>
    <models>
      <model file="sphere.3d" dayPeriod="10" />
    </models>
  </group>
</group>
```

Figure 6: Exemplo da utilização do *dayPeriod*

## 6. Conclusão

Com a conclusão da terceira fase do projeto, consideramos que conseguimos aplicar de forma eficaz os conhecimentos adquiridos sobre os conceitos em estudo, em particular as curvas de Catmull-Rom e as superfícies cúbicas de Bézier.